1 Task 2A: Instruction-Conscious L2 Replacement Policy (I-RRIP)

1.1 Goal and motivation

Server-style workloads have instruction footprints that exceed the L1I cache, so many instruction fetches are served by the L2. An L2 instruction miss stalls the front end (fetch and decode), which the out-of-order back end cannot hide, whereas many data misses can be overlapped with other work. Plain SRRIP treats both kinds of lines identically. In this task we extend SRRIP at the L2 so that instruction lines are favoured in all three parts of the policy (insertion, promotion and eviction), with the goal of reducing the L2 load MPKI of instruction lines.

1.2 Experimental setup

All results come from ChampSim with the configuration in Table 1. Every run simulates 25M warm-up instructions (caches and replacement state are trained, statistics discarded) followed by 25M measured instructions, as required by the assignment. No prefetcher is enabled at any level, and all levels except the L2 use LRU. We evaluate three L2 policies on all three provided traces (referred to as Trace1 to Trace3):

- **LRU (baseline):** the default policy with no prefetching. This is the reference for speedup.
- **SRRIP:** the provided `rrip.l2c_repl`, the policy we are improving on.
- **I-RRIP:** our instruction-conscious policy (`irrip.l2c_repl`).

Parameter	Value
Core	1 core, hashed perceptron branch predictor
L1D	64 sets × 12 ways (48 KiB), LRU
L2C	1024 sets × 8 ways (512 KiB), policy under test
LLC	2048 sets × 16 ways (2 MiB), LRU
DRAM	4096 MB, 1 channel, 6400 MT/s
Prefetchers	none at any level
Warm-up / measured	25M / 25M instructions

Table 1: Simulated configuration.

Metrics. IPC is ChampSim's cumulative IPC over the measured region. Speedup is the IPC of the policy divided by the IPC of LRU on the same trace. L2C load MPKI is the number of L2 demand-load misses per 1000 instructions, and ChampSim additionally reports it split into *instruction* and *data* load MPKI; the two add up to the total (for example I-RRIP on Trace2: $13.335 + 9.348 = 22.683$).

1.3 Design of I-RRIP

SRRIP keeps a 2-bit re-reference prediction value (RRPV, 0 to 3) per line. A new line is inserted at RRPV 2, a hit resets the line to 0, and the victim is a line with RRPV 3 (if none exists, all RRPVs in the set are incremented and the search repeats). I-RRIP changes all three steps using one extra bit per line, `is_instr`, plus the `isInstruction` flag that `cache.cc` now passes

to `update_replacement_state` (taken from the packet's `instruction` field; writebacks pass 0). Table 2 summarises the three changes.

Step	SRRIP	I-RRIP
Insertion	RRPV 2 for every line	Instruction line: RRPV 0. Data line: RRPV 2 (unchanged).
Promotion (hit)	RRPV $\rightarrow$ 0 for every line	Instruction line: RRPV $\rightarrow$ 0. Data line: RRPV decremented by one (not below 0).
Eviction	first line with RRPV 3	Among lines at RRPV 3, evict a data line first; evict an instruction line only if every line at RRPV 3 is an instruction line. If no line is at RRPV 3, age all lines and retry.

Table 2: SRRIP versus I-RRIP. Translation and other non-instruction lines are treated as data lines.

Insertion (modified). A line brought into the L2 by an instruction fetch means the L1I has just missed, so the front end is waiting on it, and the code footprint of server workloads usually fits in the L2 even when it does not fit in the L1I. We therefore insert instruction lines at RRPV 0, the highest-priority position, which gives them the longest possible time before they become eviction candidates. Data lines keep SRRIP's insertion at RRPV 2 so that streaming data is still filtered out quickly.

Promotion (modified). In SRRIP, a single hit makes a data line exactly as protected as an instruction line. Moving data lines down by only one step on a hit gives them a smaller reward, so heavily reused data cannot push all instruction lines toward eviction. Instruction lines still go directly to 0. The `is_instr` bit is set on insertion and also on any instruction hit, and is not cleared by data hits, so a line that has served instruction fetches stays protected.

Eviction (modified). Insertion and promotion only change when a line *becomes* a candidate. The victim choice is the final safeguard: when several lines have reached RRPV 3, evicting a data line first keeps instruction lines resident even after they have aged. Instruction lines are not pinned permanently. If every RRPV-3 line is an instruction line, one of them is evicted, so a set can never deadlock and a cold instruction line eventually leaves.

Why not a simpler or stronger variant. Changing insertion alone would let a data line promoted to 0 by one hit become as protected as instructions. Pinning instruction lines (never evicting them) would let stale code fill the set and starve data. The three-part bias above is a compromise that protects instructions strongly while still letting them age out. We did not run an ablation of each part individually, so the claim that each part contributes is supported by reasoning and by the eviction counters in Section 1.5, not by separate measurements.

Hardware cost. I-RRIP needs one extra bit per L2 line on top of SRRIP's 2-bit RRPV. With $1024 \times 8 = 8192$ lines this is 8192 bits (1 KiB), compared with 16384 bits (2 KiB) for the RRPVs themselves.

1.4 Results

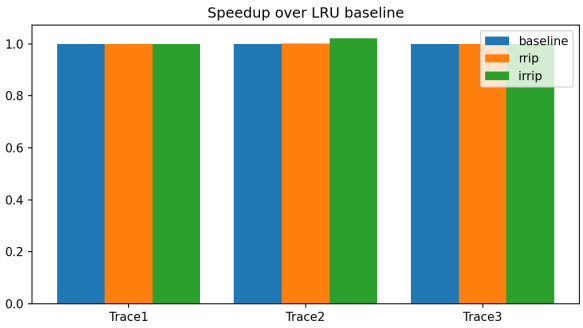
Table 3 lists the required metrics for every trace and policy (raw values are in the file `task2a_results.csv`). Table 4 adds hit rate and miss latency, and Figures 1–3 plot the same data. In the figures, the policy named "baseline" is LRU.

Trace	Policy	IPC	Speedup	L2C load MPKI	L2 instr load MPKI	L2 data load MPKI
Trace1	LRU	0.2323	1.0000	17.696	10.338	7.358
	SRRIP	0.2322	0.9995	17.639	10.562	7.077
	I-RRIP	0.2322	0.9992	17.762	9.856	7.907
Trace2	LRU	0.2512	1.0000	27.357	18.853	8.504
	SRRIP	0.2518	1.0026	25.746	17.781	7.965
	I-RRIP	0.2565	1.0211	22.683	13.335	9.348
Trace3	LRU	0.3821	1.0000	3.954	1.124	2.830
	SRRIP	0.3817	0.9991	4.039	1.197	2.842
	I-RRIP	0.3821	1.0001	3.879	0.996	2.883

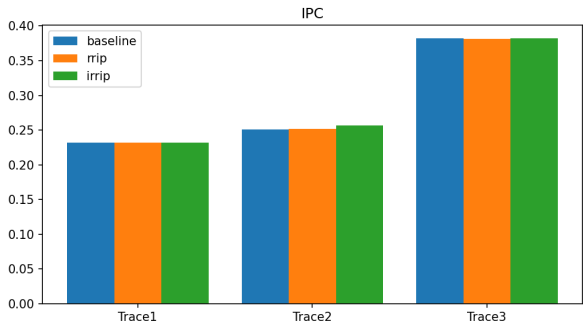
Table 3: IPC, speedup over LRU, and L2 demand-load MPKI (total, instruction, data) over the 25M measured instructions.

Trace	Policy	L2 load hit rate (%)	Avg. L2 miss latency (cycles)
Trace1	LRU	29.7	140.3
	SRRIP	29.9	140.3
	I-RRIP	29.4	134.1
Trace2	LRU	48.9	89.5
	SRRIP	51.9	93.1
	I-RRIP	57.7	93.9
Trace3	LRU	28.5	156.5
	SRRIP	26.9	151.5
	I-RRIP	29.8	154.1

Table 4: Supplementary metrics: L2 demand-load hit rate and the average L2 miss latency reported by ChampSim.

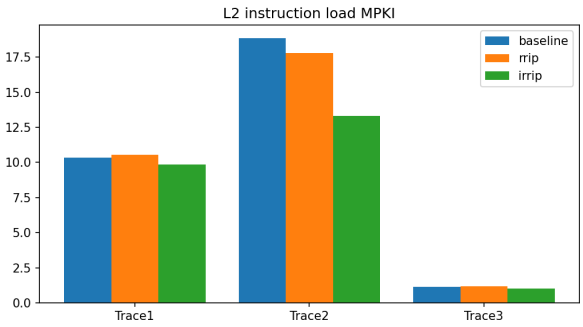


(a) Speedup over LRU.

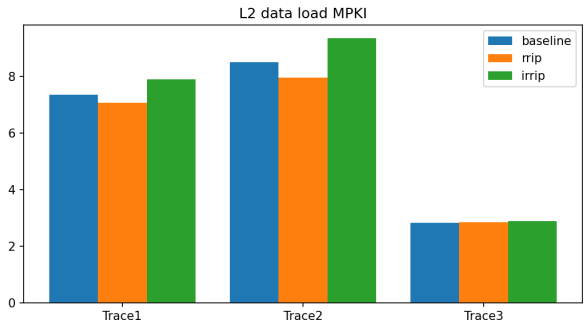


(b) IPC.

Figure 1: Performance of LRU ("baseline"), SRRIP and I-RRIP on the three traces.



(a) L2 instruction load MPKI.



(b) L2 data load MPKI.

Figure 2: L2 load MPKI split by line type.

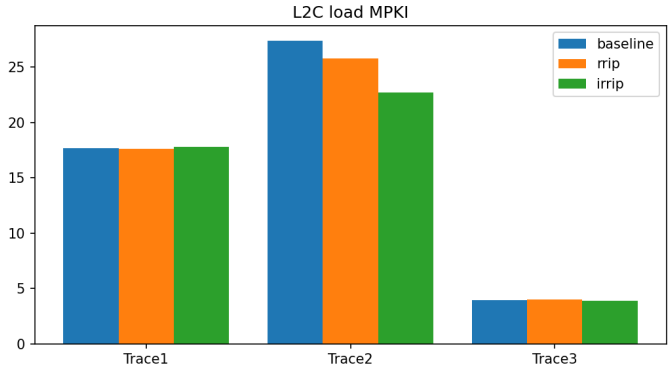


Figure 3: Total L2C load MPKI (instruction plus data).

1.5 Analysis

I-RRIP achieves its target. L2 instruction load MPKI is lower under I-RRIP than under both LRU and SRRIP on all three traces. Relative to SRRIP the reduction is 6.7% on Trace1 (10.56 → 9.86), 25.0% on Trace2 (17.78 → 13.33) and 16.8% on Trace3 (1.20 → 1.00).

Trace2: a clear gain. Trace2 has by far the highest instruction MPKI (18.9 under LRU), which is the situation I-RRIP is designed for. Instruction MPKI falls by 4.45 relative to SRRIP while data MPKI rises by only 1.38 (7.96 → 9.35), so total L2C load MPKI drops from 25.75 to 22.68 (11.9% lower than SRRIP, 17.1% lower than LRU). The L2 load hit rate rises from 48.9% (LRU) to 57.7%, and IPC improves by 2.11% over LRU and 1.85% over SRRIP. The IPC gain is much smaller than the MPKI reduction. Trace2's average L2 miss latency is only about 90 cycles, compared with about 135 to 155 on the other traces, which suggests that many of its L2 misses are served by the LLC rather than DRAM, so each avoided miss saves less time; we did not isolate this effect with a separate experiment.

Trace1: the trade-off. Instruction MPKI falls by 0.71 but data MPKI rises by 0.83 relative to SRRIP, so total MPKI is essentially unchanged (17.64 → 17.76) and so is IPC (speedup 0.9992 over LRU, 0.03% below SRRIP). The instruction gain is cancelled by extra data misses, i.e. blindly favouring instructions does not pay off when data misses cost as much as instruction misses.

Trace3: little headroom. Total L2 load MPKI is only about 4, so no replacement policy can change IPC noticeably (speedup 1.0001). I-RRIP still lowers both instruction MPKI and total MPKI (4.04 → 3.88 versus SRRIP).

SRRIP versus LRU. SRRIP gives slightly *higher* instruction MPKI than LRU on Trace1 (10.56 vs. 10.34) and Trace3 (1.20 vs. 1.12). SRRIP's scan resistance helps data, but it gives instruction lines no preference, so the instruction gains seen in I-RRIP come from the instruction-aware changes and not from RRIP itself.

Evidence from eviction counters. ChampSim prints counters of the form "L2C X Evicting Y". Reading X as the type of the incoming line and Y as the type of the resident line it displaces, Table 5 sums, for each trace, how many instruction lines and data lines were evicted (over all incoming types). We use these counters only to compare policies on the same trace. I-RRIP evicts fewer instruction lines on every trace and more data lines, which is the intended shift and matches the MPKI split. On Trace2 the number of instruction lines evicted by incoming instruction lines alone falls from 284,190 to 173,076 (39% lower), indicating that SRRIP lets instruction lines evict each other heavily on this trace.

	Instruction lines evicted			Data lines evicted		
Trace	SRRIP	I-RRIP	Δ	SRRIP	I-RRIP	Δ
Trace1	261,441	236,236	−9.6%	273,029	308,367	+12.9%
Trace2	431,648	300,977	−30.3%	289,934	358,042	+23.5%
Trace3	31,970	27,270	−14.7%	105,044	106,190	+1.1%

Table 5: Resident instruction and data lines evicted from the L2, summed over all incoming line types, from ChampSim's "L2C X Evicting Y" counters.

1.6 Limitations and link to Task 2B

The static policy helps when instruction misses dominate (Trace2), is neutral on a trace with little L2 pressure (Trace3), and loses its gain to extra data misses when the two kinds of misses are comparable (Trace1). The data MPKI increase on every trace is the price of always favouring

instructions, and it motivates the dynamic, set-dueling version in Task 2B, which can fall back to plain SRRIP in phases where prioritising instructions does not help. Our conclusions rest on three traces and a single 25M-instruction window per trace, and we did not run per-component ablations, so they should not be generalised beyond this setup.

1.7 Reproducing the results

```
./build_champsim.sh no lru   baseline
./build_champsim.sh no rrip  rrip
./build_champsim.sh no irrip irrip
bin/<binary> -warmup_instructions 25000000 \
    -simulation_instructions 25000000 \
    -traces /traces/<Trace>.gz > results/<policy>_<Trace>.txt
python3 parse.py   # writes task2a_results.csv and plot_*.png
```